

An existential layer for Basic Recursive Arithmetic: object provability, Löb’s theorem, generalised Chaitin, and the short-tower obstruction

T4 notes

2026-06-11

Abstract

We extend the quantifier-free Basic Recursive Arithmetic (BRA) of Church and Guard with a single closed existential formula former $\mathbf{E}(\mathbf{f})$, “ $\exists x. \mathbf{f}(x) = \mathbf{O}$ ”, for a unary combinator $\mathbf{f}$. Because a Fun_1 carries no term variable, every $\mathbf{E}(\mathbf{f})$ is a *closed* Σ_1 sentence, so the extension is conservative over BRA. On top of it we define an object-level provability predicate $\mathbf{Prov}(A)$, prove Löb’s theorem in the clean form $\text{Deriv}(\mathbf{Prov}(A) \supset A) \rightarrow \text{Deriv}(A)$ with *no* side condition, and prove a generalised Chaitin theorem $\mathbf{Prov}(\mathbf{D} \supset A) \Rightarrow \mathbf{Prov}(A)$ for the bounded describability predicate $\mathbf{D}$ and a *small* closed A . We then explain why the surprise–Gödel-II climb $K(x, p_r, \dots, p_N) \supset \mathbf{Prov}^r(\mathbf{O}=\mathbf{s} \mathbf{O})$ reduces, via this generalised Chaitin and a consistency-free induction, to one remaining problem: a *short* (sub-budget) formulation of $\mathbf{Prov}^r(\mathbf{O}=\mathbf{s} \mathbf{O})$. The note’s contribution is this reduction — it recasts the obstruction as a concrete *coding-length* problem rather than a consistency problem, removes the self-encoder by passing to a recogniser tower, and pins the residual $\Theta(L^*)$ wall precisely to *self-indexing* the level. We are explicit about what is and is not established: the Löb and (bounded) generalised-Chaitin results are machine-checked, the conservativity statement is a sketch, and the decisive question — whether the $\Theta(L^*)$ cost is intrinsic to consistency-freeness or only to consecutive-integer indexing — is left open.

1 The existential former $\mathbf{E}$

We work in the system recalled in [1]: terms are built from $\mathbf{O}$, variables $\mathbf{x}_i$ and the combinators $\mathbf{s}, \mathbf{o}, \mathbf{u}, \mathbf{v}, \mathbf{C}, \mathbf{R}$, the unary ones forming Fun_1 and the binary ones Fun_2 ; formulas are equations between terms, closed under $\neg$ and $\supset$; and $\text{Deriv}(A)$ denotes derivability in Guard’s Hilbert calculus. The internal verifier $\mathbf{thmT}$ is the primitive-recursive proof checker, with $\text{code}(A)$ the Gödel handle of a formula and the completeness law (necessitation, D1)

$$\text{Deriv}(A) \implies \text{Deriv}(\mathbf{thmT}(p) = \text{code}(A)) \quad \text{for a proof code } p = \text{encode}(A).$$

Definition 1.1 (the former $\mathbf{E}$). Extend the formula grammar by one clause:

$$\mathbf{A}, \mathbf{B} ::= \mathbf{t}_1 = \mathbf{t}_2 \mid \neg \mathbf{A} \mid \mathbf{A} \supset \mathbf{B} \mid \mathbf{E}(\mathbf{f}) \quad (\mathbf{f} \in \text{Fun}_1).$$

The intended reading is

$$\mathbf{E}(\mathbf{f}) \equiv \exists x. \mathbf{f}(x) = \mathbf{O}.$$

Since $\mathbf{f} \in \text{Fun}_1$ contains no term variable, $\mathbf{E}(\mathbf{f})$ is a *closed* formula: substitution is a no-op, $\mathbf{sub}(a, t, \mathbf{E}(\mathbf{f})) = \mathbf{E}(\mathbf{f})$.

Rules. We add the natural introduction and elimination rules, plus the implication form of introduction (which the elimination needs):

$$\frac{\text{Deriv}(\mathbf{f}(t) = \mathbf{O})}{\text{Deriv}(\mathbf{E}(\mathbf{f}))} \text{E-I} \qquad \frac{}{\text{Deriv}(\mathbf{f}(t) = \mathbf{O} \supset \mathbf{E}(\mathbf{f}))} \text{E-IAX}$$

$$\frac{x \text{ not free in } C \quad \text{Deriv}(\mathbf{f}(x) = \mathbf{O} \supset C) \quad \text{Deriv}(\mathbf{E}(\mathbf{f}))}{\text{Deriv}(C)} \mathbf{E-E}.$$

The eigenvariable side condition in **E-E** is automatic whenever C is itself closed (e.g. $C = \mathbf{E}(\mathbf{g})$), which is the only case we use.

Proposition 1.2 (Π_2 -conservativity, via IS_1 — sketch). *For every **E**-free Π_2 formula A ,*

$$\text{BRA} + \mathbf{E} \vdash A \implies \text{BRA} \vdash A.$$

Proof sketch. The route factors through IS_1 and needs no cut-elimination for $\text{BRA} + \mathbf{E}$.

1. *Interpretation.* Read each $\mathbf{E}(\mathbf{f})$ as the ordinary arithmetic formula $\exists x (\mathbf{f}(x) = \mathbf{O})$. The two added rules are exactly the usual rules of a decidable existential quantifier — **E-I** is $\exists$ -introduction, **E-E** is $\exists$ -elimination — and the matrix $\mathbf{f}(x) = \mathbf{O}$ is primitive recursive (indeed quantifier-free). Because the bound variable never enters an induction formula, the layer is precisely Σ_1 -existential reasoning over a p.r. predicate, so the interpretation lands inside IS_1 : $\text{BRA} + \mathbf{E}$ is interpretable in IS_1 .
2. *Parsons.* IS_1 is Π_2 -conservative over PRA.
3. *Presentation.* BRA is Church's Skolem/combinator presentation of PRA.

Hence an **E**-free Π_2 theorem A of $\text{BRA} + \mathbf{E}$ has its translation (identical, since A is **E**-free) provable in IS_1 ; by (2) it is provable in PRA, and by (3) in BRA. $\square$

Remark 1.3 (scope of the claim). We claim exactly Π_2 -conservativity over BRA for **E**-free formulas — *not* full conservativity, and *nothing* for formulas that themselves contain **E**. The non-trivial half is the interpretation step (1); Parsons' theorem alone does not give the result. We stress that the above is a *sketch*, not a verified theorem of this note: a rigorous version must spell out the interpretation of Church's combinator terms together with the **E**-rules into IS_1 , which we do not carry out here. In particular **C**, **R** remain the only recursion and the trusted base is unchanged but for the three new constructors.

2 Object provability and Löb's theorem

Definition 2.1 (provability predicate). For a formula A , let $\mathbf{f}_A \in \text{Fun}_1$ be the combinator

$$\mathbf{f}_A(x) = \mathbf{sub}(\mathbf{sO}, \mathbf{eq}(\mathbf{thmT}(x), \mathbf{code}(A))),$$

where **eq** is the internal equality test ($\mathbf{eq}(a, b) = \mathbf{sO}$ if $a = b$, else $\mathbf{O}$) and $\mathbf{sub}(\mathbf{sO}, y) = \mathbf{O}$ iff $y = \mathbf{sO}$. Thus $\mathbf{f}_A(x) = \mathbf{O}$ exactly when $\mathbf{thmT}(x) = \mathbf{code}(A)$, i.e. exactly when x is a checked proof of A . Define

$$\mathbf{Prov}(A) := \mathbf{E}(\mathbf{f}_A) \equiv \exists x. \mathbf{thmT}(x) = \mathbf{code}(A).$$

This is a closed formula, and its *code* $\mathbf{code}(\mathbf{Prov}(A))$ depends only on $\mathbf{code}(A)$ — the (large) proof witness lives behind the **E**, hidden.

Proposition 2.2 (necessitation). $\text{Deriv}(A) \implies \text{Deriv}(\mathbf{Prov}(A))$.

Proof. Take $t = \mathbf{encode}(A)$ as the existential witness. By D1, $\mathbf{thmT}(t) = \mathbf{code}(A)$, so $\mathbf{eq}(\mathbf{thmT}(t), \mathbf{code}(A)) = \mathbf{sO}$ and $\mathbf{f}_A(t) = \mathbf{sub}(\mathbf{sO}, \mathbf{sO}) = \mathbf{O}$; apply **E-I**. $\square$

Theorem 2.3 (Löb, with **Prov**). *For every formula A ,*

$$\text{Deriv}(\mathbf{Prov}(A) \supset A) \implies \text{Deriv}(A).$$

There is no freshness side condition on A .

Proof. The existing $\mathbf{thmT}$ -atom Löb (the unconditional generalisation of Guard’s clean diagonal Gödel II, see [1]) gives, for any proof-variable index k not free in A ,

$$\mathit{Deriv}\left(\left(\mathbf{thmT}(\mathbf{x}_k) = \mathit{code}(A)\right) \supset A\right) \implies \mathit{Deriv}(A).$$

It suffices to turn an $\mathbf{Prov}(A) \supset A$ hypothesis into this open-atom hypothesis. The arithmetic of $\mathbf{f}_A$ gives the object implication $(\mathbf{thmT}(\mathbf{x}_k) = \mathit{code}(A)) \supset (\mathbf{f}_A(\mathbf{x}_k) = \mathbf{O})$ (the necessitation computation, read as an implication), and $\mathbf{E}$ -Iax gives $(\mathbf{f}_A(\mathbf{x}_k) = \mathbf{O}) \supset \mathbf{Prov}(A)$. Composing,

$$(\mathbf{thmT}(\mathbf{x}_k) = \mathit{code}(A)) \supset \mathbf{Prov}(A) \supset A,$$

which is the required open-atom hypothesis. Because $\mathbf{Prov}(A)$ is closed, no condition on A is consumed by the bridge; the only index k is chosen *fresh* ($k > \max \text{variable of } A$), so the side condition of the open-atom Löb holds automatically. $\square$

3 Generalised Chaitin

Fix a budget L^* and $N = 3^{L^*+1}$ (so a number $< N$ has at most $L^* + 1$ base-3 digits). Programs *are* numbers, read by the universal decoder $\mathbf{run}$ (Church-coded): $\mathbf{run}(x, n)$ is the output of program x on fuel n . The *bounded desribability* predicate, with free program variable $\mathbf{x}_0$ and fuel variable $\mathbf{x}_1$, is

$$\mathbf{D}(\mathbf{x}_0, r, \mathbf{x}_1) := (\mathbf{x}_0 \leq L^*) \wedge (\mathbf{run}(\mathbf{x}_0, \mathbf{x}_1) = \mathbf{s} r),$$

read curried as the guard $G_A(r) := (\mathbf{x}_0 \leq L^*) \supset (\mathbf{run}(\mathbf{x}_0, \mathbf{x}_1) = \mathbf{s} r \supset A)$, so that “ $D \supset A$ ” is the L^* -bounded statement “if r is described in $\leq L^*$ digits then A ”.

Theorem 3.1 (generalised Chaitin). *Let A be a closed formula whose code is small: precisely, the diagonal program δ built below, which embeds $\mathit{code}(A)$, satisfies $\mathit{nodes}(\delta) \leq L^*$. Then for every proof code w ,*

$$\mathit{Deriv}\left(\left(\mathbf{thmT}(w) = \mathit{code}(G_A(\mathit{out} w))\right) \supset \left(\mathbf{thmT}(\delta(w)) = \mathit{code}(A)\right)\right),$$

that is, in the abbreviation $\mathbf{Prov}$,

$$\mathbf{Prov}(\mathbf{D} \supset A) \implies \mathbf{Prov}(A).$$

Construction. The diagonal δ is the short self-referential program that searches $\mathbf{thmT}(0), \mathbf{thmT}(1), \dots$ for a proof w whose conclusion is the guard G_A , reads off the subject $r = \mathit{out} w$, runs to its own output, and *fires modus ponens*. Concretely, under the hypothesis $\mathbf{thmT}(w) = \mathit{code}(G_A)$ one closes the open guard by substituting $\mathbf{x}_0 := \mathbf{num}(\delta)$, $\mathbf{x}_1 := \mathbf{num}(\mathit{fuel})$, obtaining $\mathbf{thmT}(\cdot) = \mathit{code}((\delta \leq L^*) \supset (\mathbf{run}(\delta, \cdot) = \mathbf{s} r \supset A))$, and discharges the two antecedents:

- $\delta \leq L^*$ — the *size-pin*. The diagonal genuinely is a small program: $\mathit{nodes}(\delta) \leq L^*$ gives, by the base-3 digit bound (Chaitin’s $c + \log n < n$), $\mathit{rank}(\delta) < N$, hence the object inequality $\delta \leq N$ is provable. This is where the smallness of A is consumed: δ embeds $\mathit{code}(A)$, so $\delta \leq L^*$ forces $\mathit{code}(A)$ to fit the budget.
- $\mathbf{run}(\delta, \cdot) = \mathbf{s} r$ — the *run*: the diagonal, decoded and executed, outputs its recognised subject r .

Two internalised modus ponens then give $\mathbf{thmT}(\delta(w)) = \mathit{code}(A)$.

Remark 3.2 (why the bound is essential). With an *unbounded* $\mathbf{D}$, the antecedent “ $\mathbf{run}(\mathbf{x}_0, \cdot) = \mathbf{s}r$ ” is provably true for the diagonal (it describes its own output), so $\mathbf{Prov}(\mathbf{D} \supset A) \Rightarrow \mathbf{Prov}(A)$ collapses to internalised modus ponens with a provable antecedent — trivial, and not Chaitin. The bound $\mathbf{x}_0 \leq L^*$ is exactly what makes $\mathbf{D}$ non-trivial, and then the size-pin — hence the smallness of A — is genuinely needed.

Remark 3.3 (status of the construction). In the formal development this bounded step is *discharged*, not assumed: the size-pin $\delta \leq N$ and the run $\mathbf{run}(\delta, \cdot) = \mathbf{s}r$ are *derived* from the single hypothesis $\mathit{nodes}(\delta) \leq L^*$ (the smallness of A), and the two internalised modus ponens are explicit proof-terms. The account above is a *sketch* that elides exactly those two derivations — which are the substantial part — and the high-level “ $\mathbf{Prov}(\mathbf{D} \supset A) \Rightarrow \mathbf{Prov}(A)$ ” should be read with the precise hypothesis $\mathit{nodes}(\delta) \leq L^*$ in force.

4 The climb $K \supset \mathbf{Prov}^r(\mathbf{O}=\mathbf{s} \mathbf{O})$ and the short-tower obstruction

Write $\perp := (\mathbf{O} = \mathbf{s} \mathbf{O})$ and let $\mathbf{Prov}^0(\perp) := \perp$, $\mathbf{Prov}^{r+1}(\perp) := \mathbf{Prov}(\mathbf{Prov}^r(\perp))$. The surprise–Gödel-II claim is the family, for the day- $[r..N]$ describability big conjunction $K(x, p_r, \dots, p_N)$,

$$C(r) \equiv \text{Deriv} \left(K(x, p_r, \dots, p_N) \supset \mathbf{Prov}^r(\perp) \right).$$

Base. $C(0)$ is $K(x, p_0, \dots, p_N) \supset \perp$, i.e. $\neg K$ over all $N+1$ days, which is the pure pigeonhole (more days than short programs) — consistency-free.

Induction, consistency-free. For $C(r) \rightarrow C(r+1)$ one proceeds exactly as in the surprise step, but replacing the ex-falso target $\perp$ by $\mathbf{Prov}^r(\perp)$ and replacing the role of the global consistency hypothesis by the induction hypothesis $C(r)$. Concretely: under $K(x, p_{r+1}, \dots, p_N)$, the extra-day describability $\mathbf{D}(g, r)$ together with the rest gives $K(x, p_r, \dots, p_N)$, whence $C(r)$ yields $\mathbf{Prov}^r(\perp)$; so the bounded guard $\mathbf{D} \supset \mathbf{Prov}^r(\perp)$ is *provable*, and Theorem 3.1 at $A := \mathbf{Prov}^r(\perp)$ fires it — with *no* consistency clash — to $\mathbf{Prov}(\mathbf{Prov}^r(\perp)) = \mathbf{Prov}^{r+1}(\perp)$, i.e. $C(r+1)$. The induction hypothesis $C(r)$ plays the role the old day-clash assigned to Con; since $C(0)$ is the pigeonhole, no global consistency is used anywhere.

The single remaining problem. The induction invokes Theorem 3.1 at $A = \mathbf{Prov}^r(\perp)$, whose hypothesis is

$$\mathit{nodes}(\delta_r) \leq L^*, \quad \delta_r \text{ the diagonal embedding } \text{code}(\mathbf{Prov}^r(\perp)).$$

The literal nested $\mathbf{Prov}^r(\perp)$ has code of size $\Theta(r) \approx \Theta(N)$, busting the budget at once. A *small-coded* tower fixes the functional dependence on r : one encodes $\mathbf{Prov}^r(\perp)$ not as the nested formula but as a fixed closed sentence $\mathbf{E}(g)$ whose combinator g carries the compact base-3 numeral r and recognises the previous level, giving

$$\text{code}(\mathbf{Prov}^r(\perp)) = \text{Const} + O(\log r).$$

This is small *as a function of* r (the literal tower is $\Theta(r)$); but it is *not* small relative to the budget. Since $r \leq N = 3^{L^*+1}$, for r near N the numeral alone has $\sim L^*$ base-3 digits, i.e.

$$\text{code}(\mathbf{Prov}^r(\perp)) = \Theta(L^*),$$

and the budget *is* L^* . Naming the index r already saturates the budget: the diagonal cannot be strictly smaller than the level r it must encode, and r is sized to the budget itself. This is the precise asymmetry with ordinary Chaitin, where the diagonal encodes the *threshold* L^* in $O(\log L^*)$ digits, not an index up to N . Re-defining L^* from the r -th diagonal makes N

vary with r and breaks the shared N of the family $C(r)$; and the alternative of reading r as a runtime witness rather than embedding $\text{code}(\mathbf{Prov}^r(\perp))$ requires an object code-builder for one **Prov**-layer, which is the impossible self-encoder (a node and a leaf would have to receive the two distinct double-codes a structural fold demands).

Thus the climb $K(x, p_r, \dots, p_N) \supset \mathbf{Prov}^r(\perp)$ reduces, with everything else in place (the existential layer, **Prov**, Löb, generalised Chaitin, the consistency-free induction), to a single open problem:

Find a formulation of $\mathbf{Prov}^r(\perp)$ whose code is strictly sub-budget, of size $o(L^)$ rather than $\Theta(L^*)$, for all $r \leq N = 3^{L^*+1}$.*

5 A possible way forward: short notations, and its limit

The obstruction is about the *absolute* size of $\text{code}(\mathbf{Prov}^r(\perp))$, which is $\Theta(L^*)$ because the index r is written out. One would like a *short notation*: bake into the formula not the numeral r but a short program e with $\text{eval}(e) = r$, and set

$$P_e := \mathbf{E}(g_e), \quad g_e(w) = \mathbf{O} \iff \mathbf{thmT}(w) \text{ witnesses stage } \text{eval}(e) \text{ of the tower.}$$

Then $\text{code}(P_e) = \text{Const} + O(|e|)$ rather than $\text{Const} + O(\log r)$.

The catch, and how the recognition tower meets it. A short notation helps only if the logic can *interpret* it when forming the code, i.e. g_e must compute/recognise the $\text{eval}(e)$ -th stage *without carrying that stage in its own code*:

g_e computes/recognises $\text{tower}(\text{eval}(e))$ without carrying the tower in $\text{code}(g_e)$.

This is exactly what a *recogniser* (as opposed to a code-builder) does. In the shipped recognition tower one sets $\text{Tower}(\mathbf{O}) = \perp$ and $\text{Tower}(r+1) = \mathbf{E}(\text{gFun}_r)$, where gFun_r carries only a *compact label* (a base-3 numeral) and checks a fixed *Fst/Snd* projection of $\mathbf{thmT}(w)$ against it — it never reconstructs $\text{code}(\text{Tower}(r))$. Replacing the stored numeral by “read e , run the fixed evaluator to $r = \text{eval}(e)$, form the comparison numeral, compare” keeps $\text{code}(g_e) = \text{Const} + O(|e|)$: the evaluator and the numeral-former are *fixed* constants, and the long label r is produced at *run time*, not stored. So the boxed condition is met, and the dangerous code-builder $c \mapsto \text{code}(\mathbf{Prov}(A_c))$ — the impossible self-encoder — never appears.

The bridge, one direction only. The surprise step needs “ $P_{e_{r+1}}$ behaves as $\mathbf{Prov}(P_{e_r})$ ”. The generalised-Chaitin step delivers a $\mathbf{thmT}(\delta) = \text{code}(P_{e_r})$, i.e. a $\mathbf{Prov}(P_{e_r})$ -witness, and the recogniser turns it into the next stage:

$$\text{Deriv}(\mathbf{thmT}(w) = \text{code}(\text{Tower}(r))) \implies \text{Deriv}(\text{Tower}(r+1))$$

holds (it is the recogniser firing under the trivially-true hypothesis). This is the *climb-up* direction, and it is all the induction uses. The *converse* $\text{Deriv}(\text{Tower}(r+1)) \rightarrow \text{Deriv}(\mathbf{Prov}(\text{Tower}(r)))$ would need recogniser *soundness* (passing the projection implies $\mathbf{thmT}(w) = \text{code}(\text{Tower}(r))$), which the recognition tower does not prove: it is complete-only, recognise $\neq$ reconstruct. Hence $P_{e_{r+1}}$ is a genuinely *weaker, short-coded surrogate* for $\mathbf{Prov}(P_{e_r})$ — entailed by it, but not equivalent to it.

Where the difficulty re-concentrates. With recognition the bridge and the self-encoder are dealt with; the whole question becomes the length $|e_r|$ of the notation, over the *whole* climb $r = 0, 1, \dots, N$. For the top, compressible, level this is favourable: $N = 3^{L^*+1}$ itself has a notation e_N of length $O(\log L^*)$. But the induction over the consecutive-day conjunction $K(x, p_r, \dots, p_N)$ traverses *every* $r \leq N$, and a generic — Kolmogorov-incompressible — index r has no notation shorter than $\Theta(\log r) = \Theta(L^*)$, so the intermediate stages still bust the budget. Re-indexing to skip the incompressible levels conflicts with the consecutive-day form of K (and an incremental $e_{r+1} = \text{succ}(e_r)$ makes $|e_r|$ grow with r). Thus the recognition/short-notation route *relocates* the obstruction — from a self-encoder to the notation length — but does not, by itself, dissolve it: the missing ingredient is a notation system, or a restructured induction, under which *every* stage actually used has a sub-budget code.

6 Is the $\Theta(L^*)$ bound intrinsic?

The value of the diagnosis in Sections 4–5 is that it turns the failure into one sharp question: is the $\Theta(L^*)$ cost of a positive stage *intrinsic*, or an artefact of how the climb is indexed?

An information-theoretic lower bound — for self-indexing. There are $N+1 = 3^{L^*+1}+1$ levels. Any stage formula that must *distinguish its own level* r from the others carries at least $\log(N+1) = \Theta(L^*)$ bits of “which level” information, so its code is $\Omega(L^*)$ for generic r — whether recognised or reconstructed, short-named or numeral-named, tag-separated or not. Hence the bound *is* intrinsic to any *self-indexing* positive stage; the single obstruction is that a self-identifying late level costs the whole budget.

But not intrinsic to the climb. Crucially, the descent — the original surprise proof, using Con — shows a level need *not* be self-identified: its per-stage conclusion is the *constant code* $(\perp)$, and “which day” lives only in the *hypothesis* $K(x, p_r, \dots, p_N)$, where the budget does not apply. So the $\Theta(L^*)$ cost is an artefact of demanding that each *positive* stage name its own level, not of the climb as such; it is a coding-length phenomenon, not a consistency one.

The decisive test (direction 3). This isolates the real mathematical question. Seek an *index-free* positive surrogate: a single, fixed, sub-budget sentence S whose intended reading at day r is fixed by the surrounding conjunction $K(x, p_r, \dots, p_N)$ rather than by a numeral baked into $\text{code}(S)$, such that the consistency-free generalised-Chaitin step still climbs to it. If such an S exists, the climb goes through within budget and yields a consistency-free positive Gödel II. If, instead, every consistency-free positive surrogate provably *must* self-identify its level, then the $\Theta(L^*)$ wall is genuinely intrinsic to consistency-freeness, and the descent (with Con) is the only budget-respecting route. Settling this dichotomy — not Löb or generalised Chaitin themselves — is the mathematical content this note leaves open.

Status of this note. As a finished paper this is not complete: Proposition 1.2 is a sketch, and the generalised-Chaitin construction is stated above the level of its size/run derivations (which are discharged formally). As a research note its contribution is the reduction itself — recasting the surprise/Gödel-II obstruction as a concrete *coding-length* problem (represent $\mathbf{Prov}^r(\perp)$, or an index-free surrogate, below budget) rather than a consistency problem, with the self-encoder removed by recognition and the remaining wall pinned to self-indexing.

References

- [1] T4 notes, *A road to Gödel's second incompleteness theorem in Basic Recursive Arithmetic* (`arGII.tex`).
- [2] J. Guard, *Automated Logic and the recursive functions* (Church's Basic Recursive Arithmetic), 1963.
- [3] A. Church, *The calculi of lambda-conversion* / Princeton lectures on recursive arithmetic.
- [4] C. Parsons, *On a number-theoretic choice schema and its relation to induction*, in Intuitionism and Proof Theory, 1970 ($\text{I}\Sigma_1$ is Π_2 -conservative over PRA).